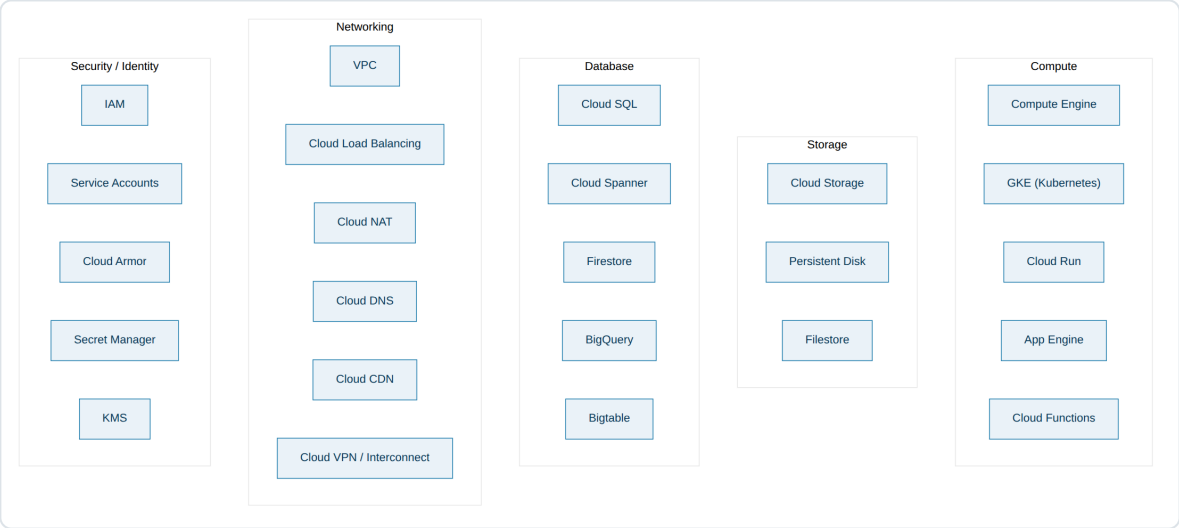
GCP Concepts Notes

GCP Services (Overview)

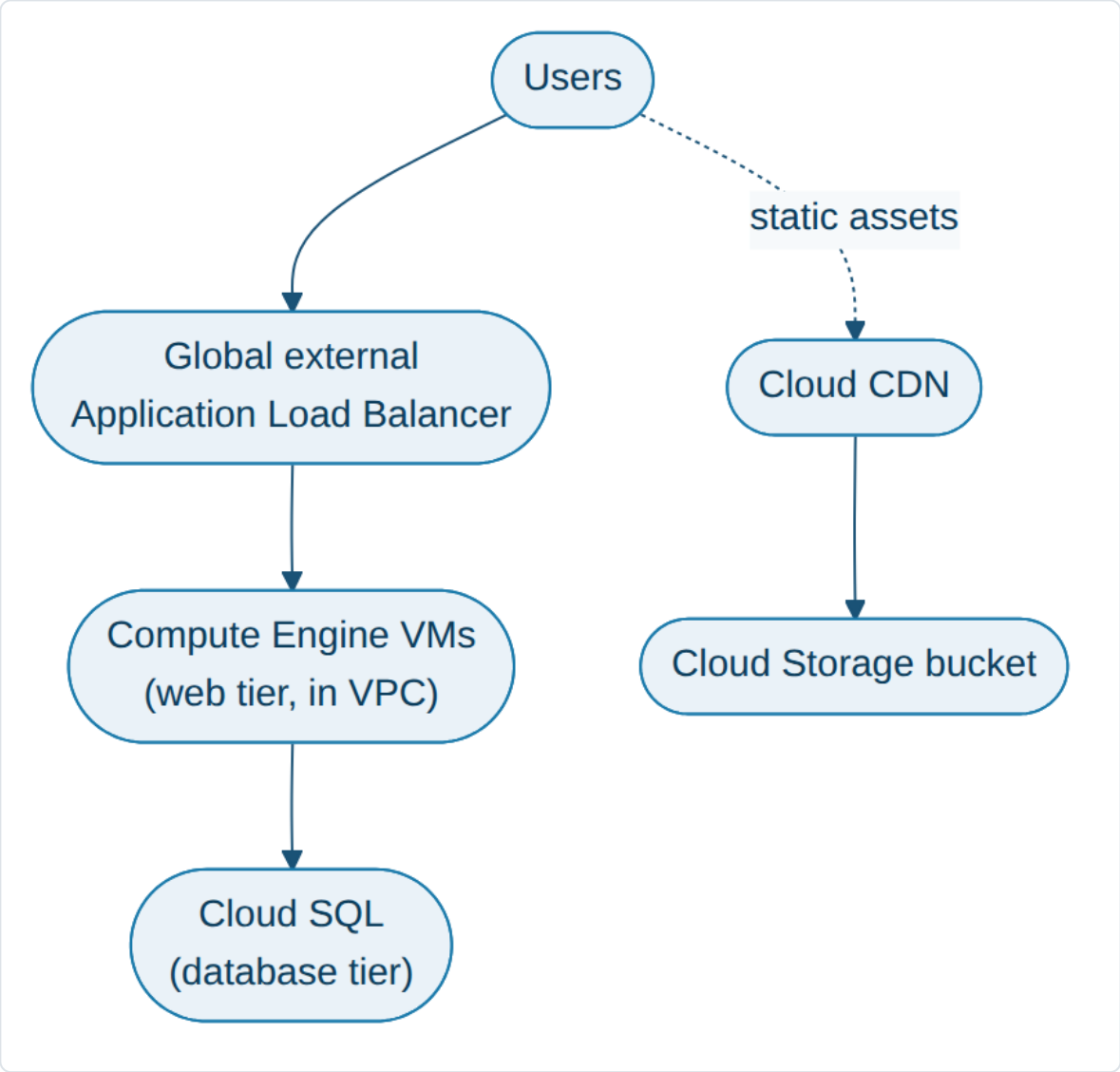
- GCP → Google Cloud Platform → offers 200+ managed services grouped into categories.
- You rarely use one service alone → a typical app combines Compute + Networking + Database + Security services together.



- Compute → Compute Engine, GKE (Kubernetes), Cloud Run, App Engine, Cloud Functions.
- Storage → Cloud Storage (buckets), Persistent Disk, Filestore.
- Database → Cloud SQL, Cloud Spanner, Firestore, BigQuery, Bigtable.
- Networking → VPC, Cloud Load Balancing, Cloud NAT, Cloud DNS, Cloud CDN, Cloud VPN/ Interconnect.
- Security / Identity → IAM, Service Accounts, Cloud Armor, Secret Manager, KMS.
- DevOps → Cloud Build, Artifact Registry, Cloud Deploy.
- Observability → Cloud Monitoring, Cloud Logging, Cloud Trace.

Example → Simple 3-tier web app on GCP

- Users → hit a **Global external Application Load Balancer**.
- LB forwards to → **Compute Engine VMs** (web tier) inside a **VPC**.
- Web tier calls → **Cloud SQL** (database tier) using its **Service Account**.
- Static assets (images, JS, CSS) served from → **Cloud Storage** + **Cloud CDN**.



Compute Engine

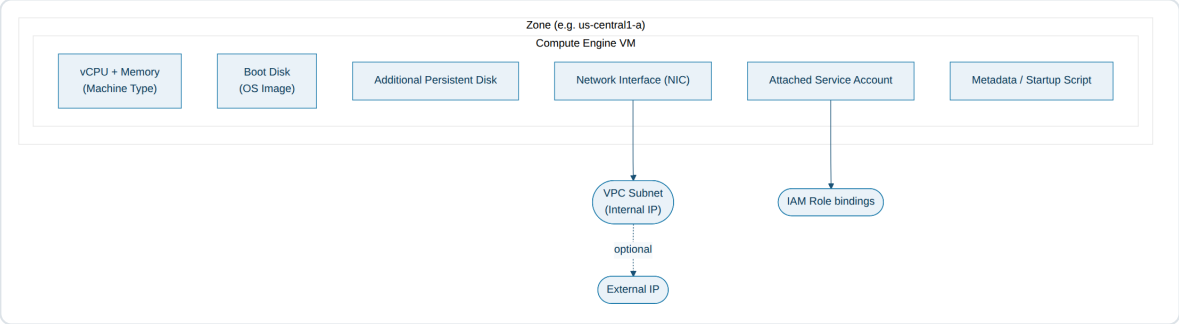
- Compute Engine → GCP's IaaS (Infrastructure as a Service) → lets you create & run Virtual Machines (VMs) on Google's infrastructure, with full control over the OS.
- You choose → Machine type (vCPU & Memory), Boot disk (OS image), Region & Zone, Network (VPC + Subnet), Service Account.
- Billing → per-second billing (after a 1 minute minimum); Sustained-use & Committed-use discounts reduce cost for long-running workloads.
- Machine families →
- `E2` → cost-optimized, general purpose.
- `N2 / N2D` → balanced price-performance.
- `C2 / C2D` → compute-optimized (high performance).
- `M1 / M2` → memory-optimized.

- `A2 / G2` → accelerator-optimized (GPU workloads).
- Images → Public images (Debian, Ubuntu, Windows, etc.), Custom images (your own baked OS + software), or Container-Optimized OS (to run containers directly on a VM).

VM (Virtual Machine)

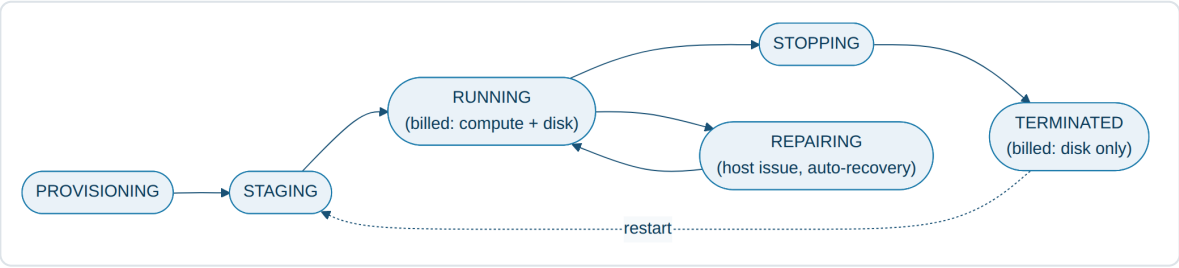
- VM → a single compute instance running inside Compute Engine.
- Each VM has →
- vCPU & Memory (based on machine type)
- Boot disk (Persistent Disk / SSD)
- one or more Network Interfaces (NIC) → attached to a VPC subnet
- Internal IP (always) & External IP (optional)
- a Service Account attached (for calling other GCP APIs)
- Metadata & Startup scripts (run automatically on boot — used to install software, pull configs, etc.)

VM anatomy diagram



VM lifecycle

- A VM moves through well-defined states — useful for understanding billing & availability:
- `PROVISIONING` → resources are being reserved.
- `STAGING` → resources reserved, VM is booting.
- `RUNNING` → VM is up (billed for compute + disk).
- `STOPPING / TERMINATED` → VM shut down (only disk storage billed, not compute).
- `REPAIRING` → GCP is auto-recovering the VM after a host issue.



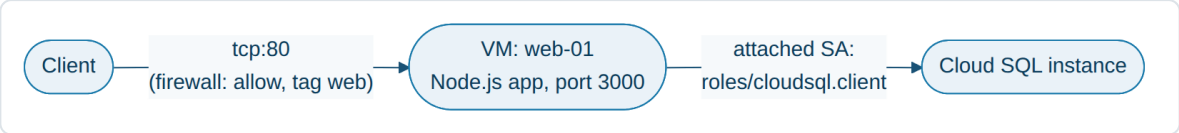
Instance Groups & Autoscaling

- Managed Instance Group (MIG) → a group of identical VMs created from an Instance Template.
- Supports → Autoscaling (add/remove VMs based on CPU/load), Auto-healing (recreate unhealthy VMs via health checks), Rolling updates, Load balancing across zones.
- Unmanaged Instance Group → group of dissimilar VMs (no autoscaling/auto-healing) — mainly used to put existing, non-identical VMs behind a load balancer.

```
gcloud compute instances create my-vm \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --network=my-vpc \
  --subnet=my-subnet \
  --service-account=my-sa@project.iam.gserviceaccount.com
```

Example → Hosting a Node.js web app on a single VM

- Create VM `web-01` in `us-central1-a`, machine type `e2-medium`, boot disk = Debian 12.
- Startup script installs Node.js & pulls app code from a repo, then runs `npm start` on port `3000`.
- Firewall rule allows `tcp:80` from `0.0.0.0/0` (via a reverse proxy/LB) into the VM's network tag `web`.
- VM's Service Account has `roles/cloudsql.client` so the app can connect to Cloud SQL.

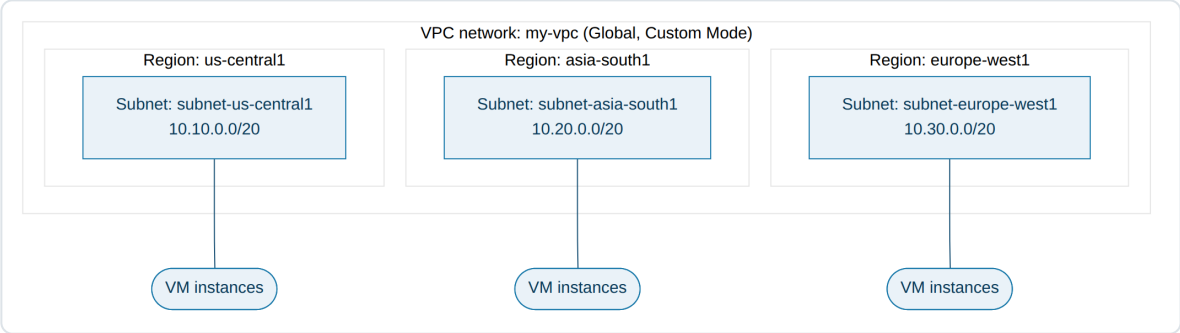


VPC (Virtual Private Cloud)

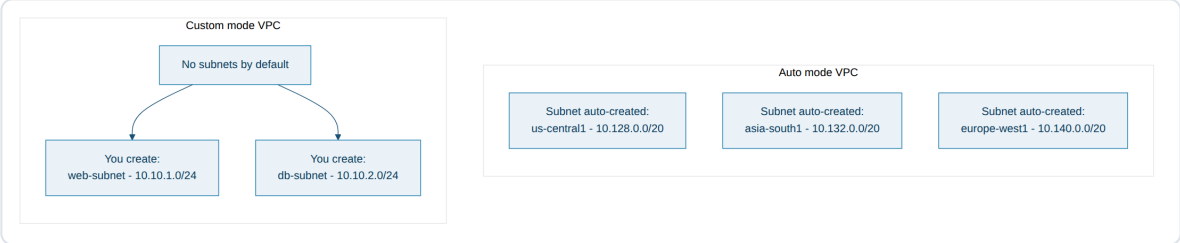
- VPC → a private, global, software-defined network that provides connectivity for your GCP resources (VMs, GKE, App Engine flex, etc).
- VPC is a **Global resource** → spans all GCP regions, but Subnets inside it are **Regional resources**.
- Two modes →
- **Auto mode** → GCP automatically creates one subnet per region (using a fixed `10.128.0.0/9` range). Good for quick start / learning.
- **Custom mode** → you manually create subnets, choosing region & CIDR range yourself. Recommended for production (full control, avoids IP overlap).
- A VPC has no IP range of its own → the IP ranges live on its subnets.

- Routes → every VPC has default routes for the internet gateway (`0.0.0.0/0`) and for each subnet's IP range; custom static routes can be added.
- A VM can only talk to resources whose subnet is reachable via VPC routing + allowed by firewall rules — both conditions must be true.

VPC architecture diagram

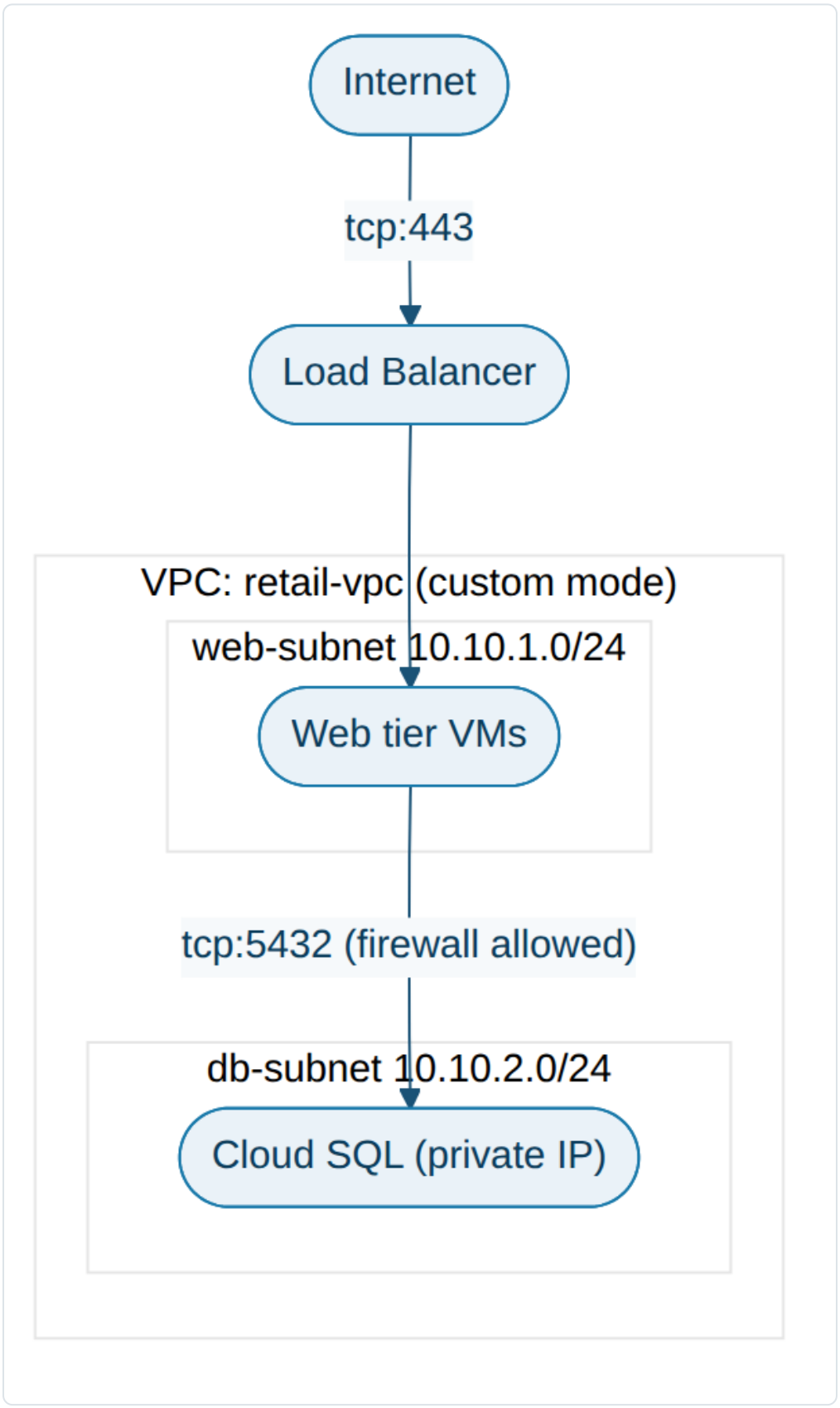


Auto mode vs Custom mode



Example → 2-tier VPC for a retail app

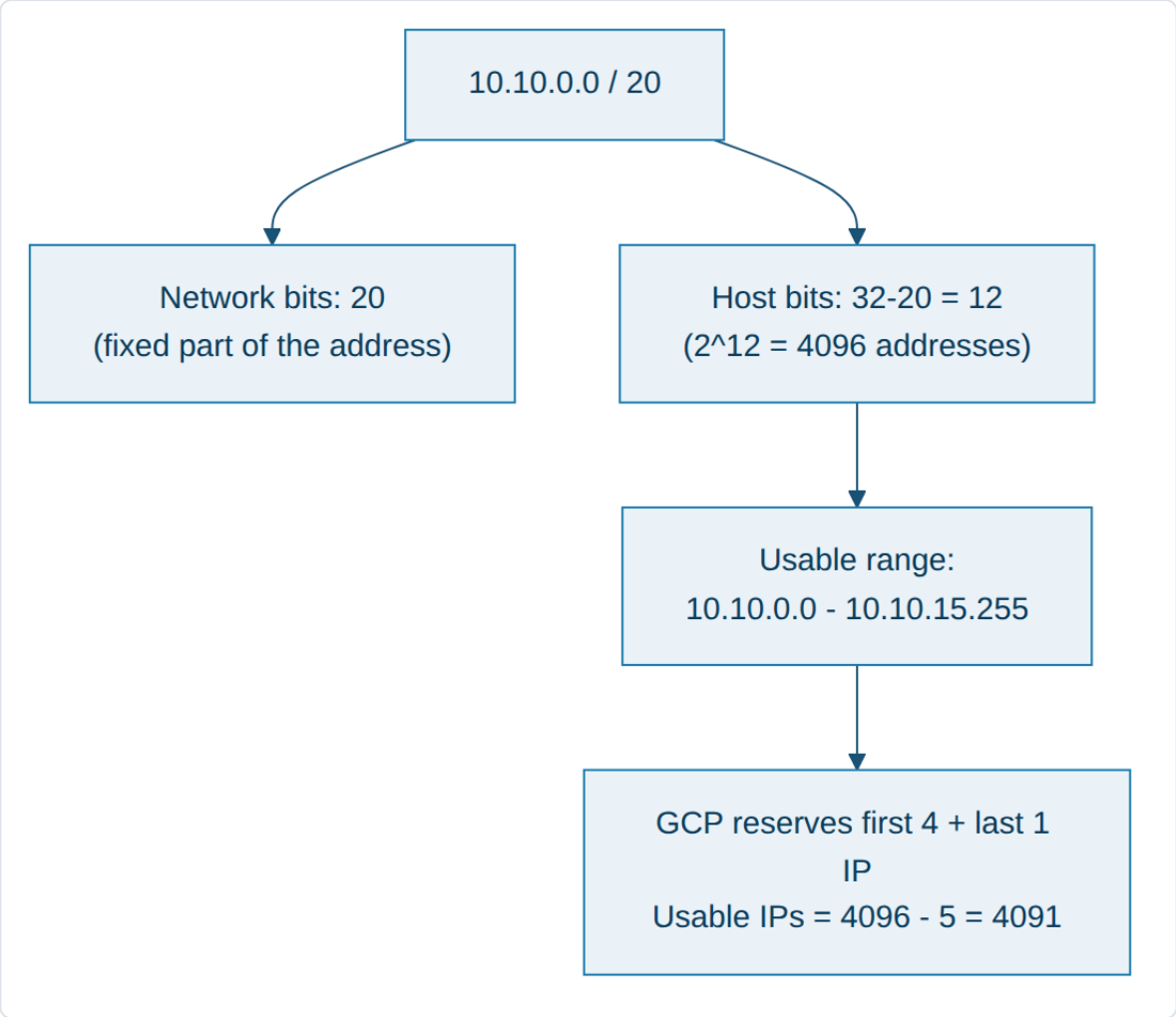
- VPC `retail-vpc` (custom mode).
- Subnet `web-subnet` → `us-central1` → `10.10.1.0/24` → holds the web-tier VMs.
- Subnet `db-subnet` → `us-central1` → `10.10.2.0/24` → holds the Cloud SQL private IP / DB VMs.
- Firewall → `web-subnet` can talk to `db-subnet` on `tcp:5432` only; internet can only reach `web-subnet` on `tcp:443` via the load balancer.


Internet
tcp:443
Load Balancer
VPC: retail-vpc (custom mode)
web-subnet 10.10.1.0/24
Web tier VMs
tcp:5432 (firewall allowed)
db-subnet 10.10.2.0/24
Cloud SQL (private IP)

Subnet & CIDR Range

- Subnet → a regional IP address range inside a VPC network. Resources (VMs) in a zone use a subnet from that zone's region.
- Every subnet has →
- a **Primary IPv4 range** → used by VM NICs, internal LB forwarding rules.
- optional **Secondary IPv4 ranges** → used only for Alias IP ranges (commonly used by GKE Pods/Services ranges).
- CIDR → Classless Inter-Domain Routing → notation `A.B.C.D/n` where `n` = number of network bits (the rest are host bits).
- Smaller `/n` → bigger range (more usable IPs). Bigger `/n` → smaller range.
- Eg → `/24` = 256 IPs, `/22` = 1024 IPs, `/20` = 4096 IPs, `/16` = 65536 IPs.
- GCP reserves the **first 4 and the last 1** IP of every subnet range (network, gateway, broadcast reserved, etc.) → usable IPs = total − 5.
- Subnets can be expanded later (increase the `/n` → smaller number) without recreating the VPC, as long as the new range doesn't overlap another subnet.

CIDR breakdown example

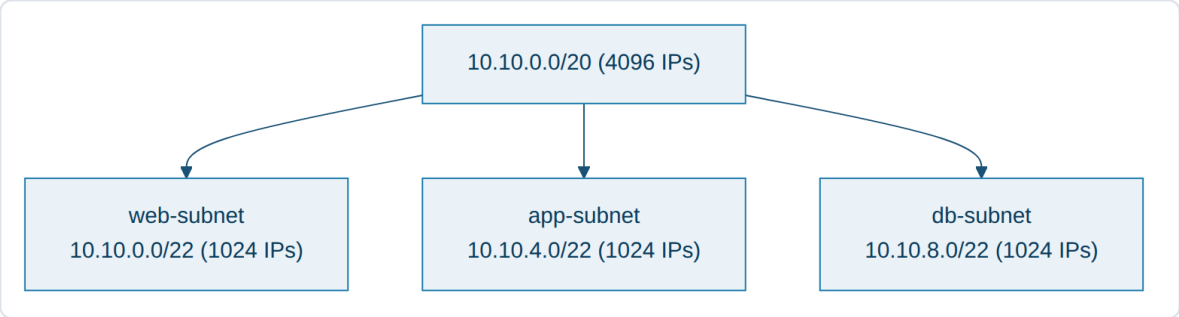



```
VPC: my-vpc (global, custom mode)

Subnet: subnet-us-central1   region=us-central1   range=10.10.0.0/20
(4096 IPs)
Subnet: subnet-asia-south1   region=asia-south1    range=10.20.0.0/20   (409
6 IPs)
Subnet: subnet-europe-west1  region=europe-west1   range=10.30.0.0/20   (409
6 IPs)
```

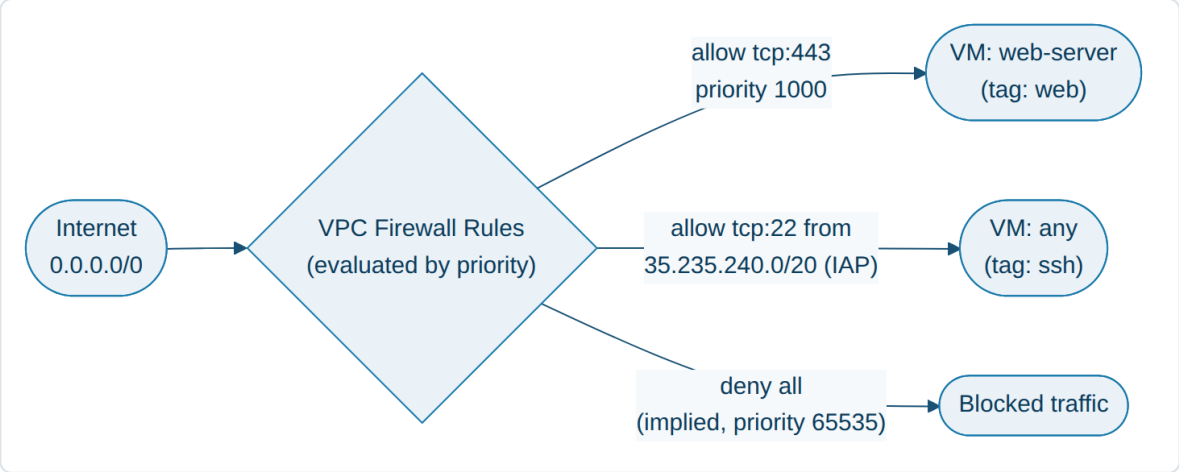
Example → Splitting one range into 3 tiers

- Start with a single block `10.10.0.0/20` (4096 IPs) reserved for `us-central1`.
- Split it into 3 smaller subnets, one per application tier:
- `web-subnet` → `10.10.0.0/22` (1024 IPs)
- `app-subnet` → `10.10.4.0/22` (1024 IPs)
- `db-subnet` → `10.10.8.0/22` (1024 IPs)

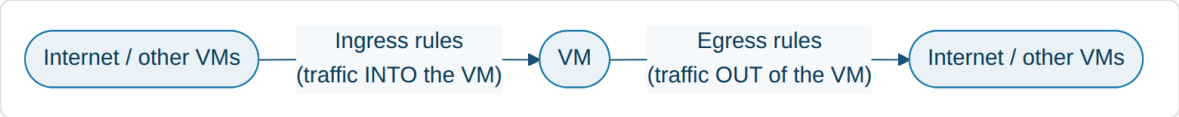


Firewall Rules

- Firewall rules → control ingress/egress traffic to/from VM instances. Applied at the VPC level (stateful — return traffic for an allowed connection is automatically allowed).
- Every VPC has 2 implied rules → `deny all ingress` and `allow all egress` (lowest priority, 65535).
- A firewall rule has →
- Direction → Ingress / Egress
- Action → Allow / Deny
- Target → all instances, a Target tag, or a Service Account
- Source/Destination → IP range or another tag
- Protocols & Ports → tcp:22, tcp:443, etc.
- Priority → 0 (highest) to 65535 (lowest); lower number wins; ties broken by "deny wins" is false — the more specific/lower priority rule wins.
- Common rules → allow-ssh (tcp:22 from IAP range `35.235.240.0/20`), allow-http/https (tcp: 80/443 from `0.0.0.0/0`), allow-internal (all protocols within subnet CIDR).



Ingress vs Egress direction



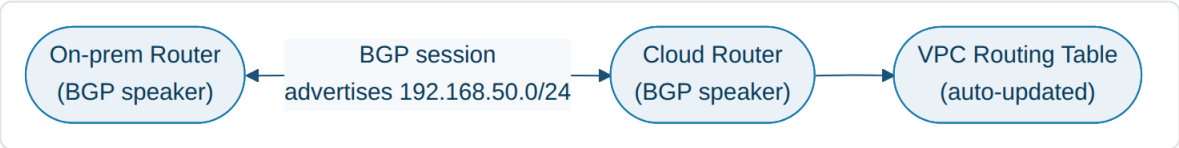
Example → Locking down a 3-tier app

- `allow-lb-health-check` → Ingress, allow tcp:80 from `130.211.0.0/22` and `35.191.0.0/16` (Google's health-check ranges) → target tag `web`.
- `allow-iap-ssh` → Ingress, allow tcp:22 from `35.235.240.0/20` → target tag `web` (SSH only via Identity-Aware Proxy, never from the open internet).
- `allow-web-to-db` → Ingress, allow tcp:5432 from tag `web` → target tag `db`.
- `deny-everything-else` → implied default deny (priority 65535) blocks anything not explicitly allowed.

Cloud Router

- Cloud Router → a fully-managed BGP (Border Gateway Protocol) speaker; it is the **control plane** required by:
- Cloud NAT (to allocate NAT IPs/ports).
- Cloud VPN (dynamic/HA routing).
- Cloud Interconnect (dynamic route exchange with on-prem).
- Cloud Router itself does not carry any user traffic — it only exchanges routes.
- With **Dynamic routing** (via Cloud Router/BGP), route changes on-prem (e.g. a new subnet added) are learned automatically — no manual static route updates needed on either side.

Cloud Router BGP peering diagram



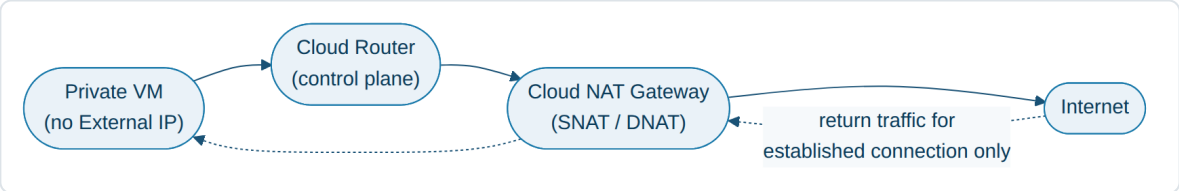
Example → Advertising a new on-prem subnet

- On-prem network `192.168.0.0/16` is connected to GCP via Cloud VPN + Cloud Router.
- Ops team adds a new on-prem subnet `192.168.50.0/24`.
- On-prem BGP router advertises the new route → Cloud Router learns it automatically → VPC routing table updates → GCP VMs can now reach `192.168.50.0/24` with zero manual changes on the GCP side.

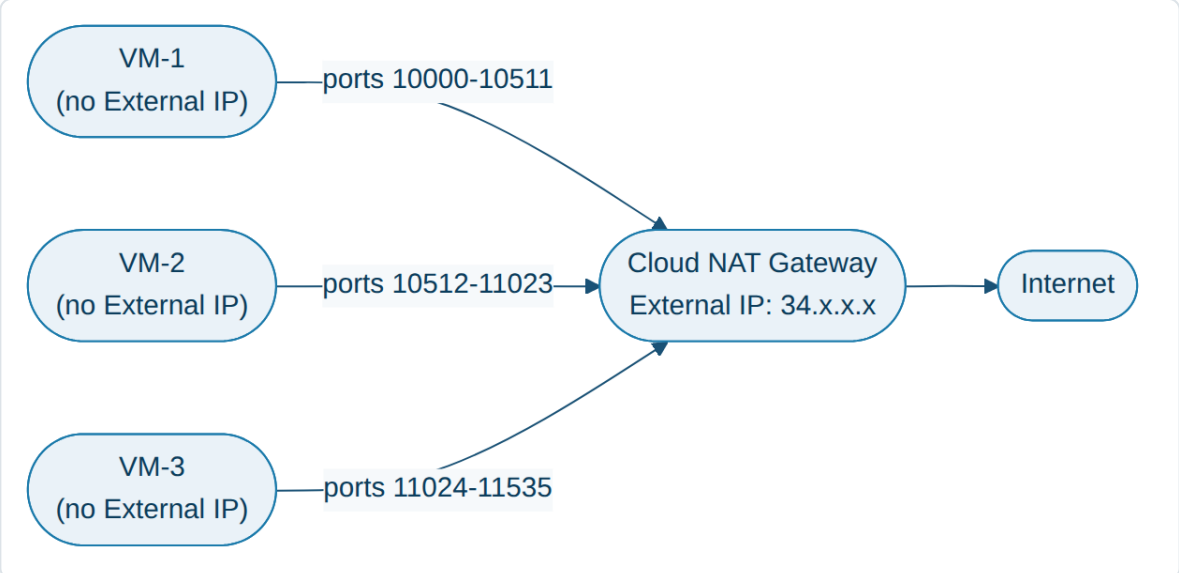
Cloud NAT

- Cloud NAT → Google's managed **Network Address Translation** service.
- Purpose → gives VMs that have **no External IP** the ability to reach the internet (outbound only) — e.g. to download OS patches, call external APIs.
- Cloud NAT is regional & requires a Cloud Router in the same VPC + region.
- It does **NOT** allow unsolicited inbound connections from the internet (no ingress) — outbound-initiated only.
- NAT allocates external IP + port ranges per VM dynamically (SNAT on the way out, DNAT on the way back for that connection).
- Minimum ports per VM is configurable — if a VM opens more concurrent connections than its allotted ports, it gets **NAT port exhaustion** errors; increasing "Minimum ports per VM" or adding more NAT IPs fixes this.

Cloud NAT flow diagram



Port allocation diagram



```
gcloud compute routers create my-router --network=my-vpc --region=us-
central1

gcloud compute routers nats create my-nat \
  --router=my-router \
  --region=us-central1 \
  --nat-all-subnet-ip-ranges \
  --auto-allocate-nat-external-ips
```

Example → 50 backend VMs need OS patches, no public IPs

- 50 VMs in `app-subnet` (no External IP, by design — reduces attack surface).
- Cloud NAT gateway `my-nat` attached to Cloud Router `my-router` in `us-central1`.
- Each VM is dynamically assigned a slice of ports on one of the NAT gateway's external IPs.
- VM runs `apt-get update && apt-get upgrade` → traffic goes VM → Cloud Router/NAT → internet → package mirror → response flows back through the same NAT mapping.
- No inbound rule needed/possible from the internet to these VMs — only their own outbound-initiated responses return.

Load Balancer

- Cloud Load Balancing → fully-managed, software-defined load balancing at Google's network edge. Two main dimensions decide the type: **Global vs Regional**, and **Layer 7 (HTTP/S) vs Layer 4 (TCP/UDP)**.

Types

Application Load Balancers (Layer 7 / HTTP(S)):

- Global external Application Load Balancer → internet-facing, multi-region, Layer-7 routing (host/path rules), uses `0.0.0.0/0`-style anycast IP.
- Regional external Application Load Balancer → internet-facing, single region.
- Regional internal Application Load Balancer → for clients inside the same VPC (or peered/VPN/Interconnect-connected network) — no internet exposure.
- Cross-region internal Application Load Balancer → internal HTTP(S) traffic spanning multiple regions.

Proxy Network Load Balancers (Layer 4 / TCP with proxy, optional SSL offload):

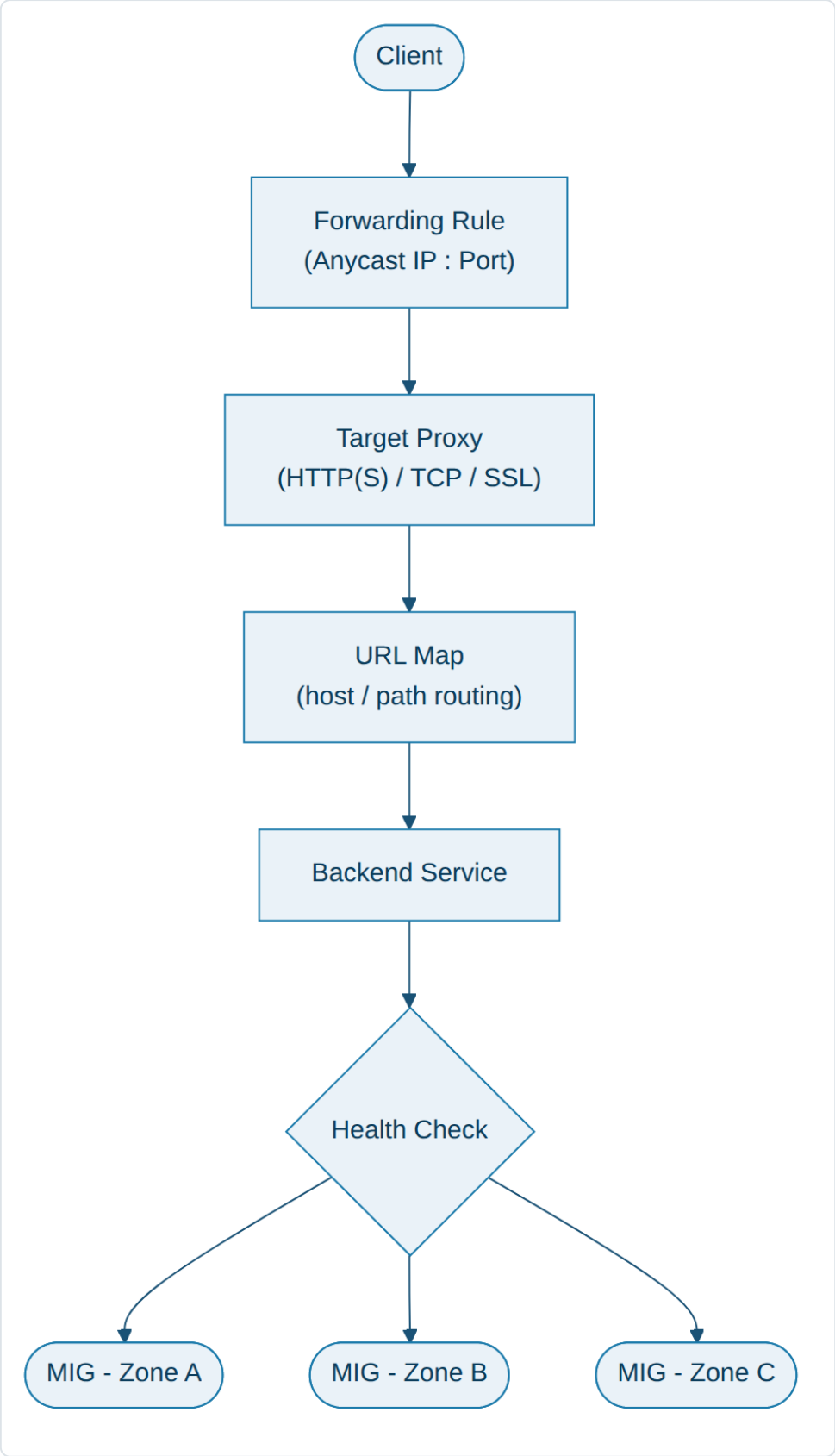
- Global external Proxy Network Load Balancer → TCP/SSL proxy, backends in multiple regions.
- Regional external / internal Proxy Network Load Balancer → same, scoped to one region.

Passthrough Network Load Balancers (Layer 4, preserves client source IP, no proxy overhead):

- Global / Regional external Passthrough Network Load Balancer → supports TCP, UDP, ESP, ICMP for internet clients.

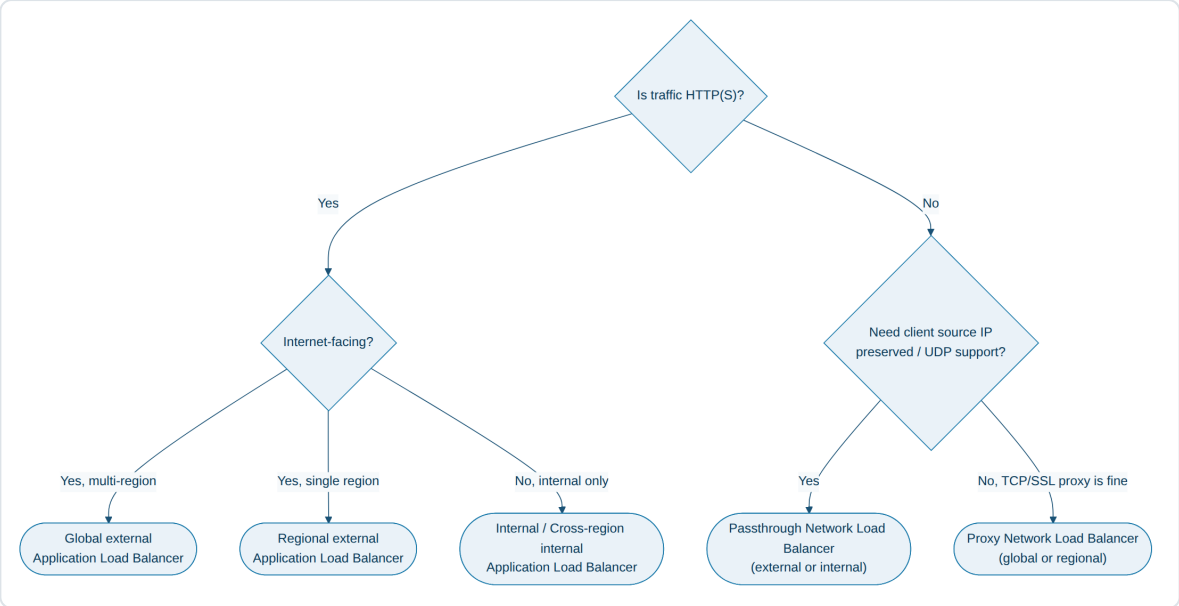
- Internal Passthrough Network Load Balancer → regional only, for internal TCP/UDP/ICMP/ESP/GRE traffic (classic "Internal LB").

Load Balancer architecture diagram



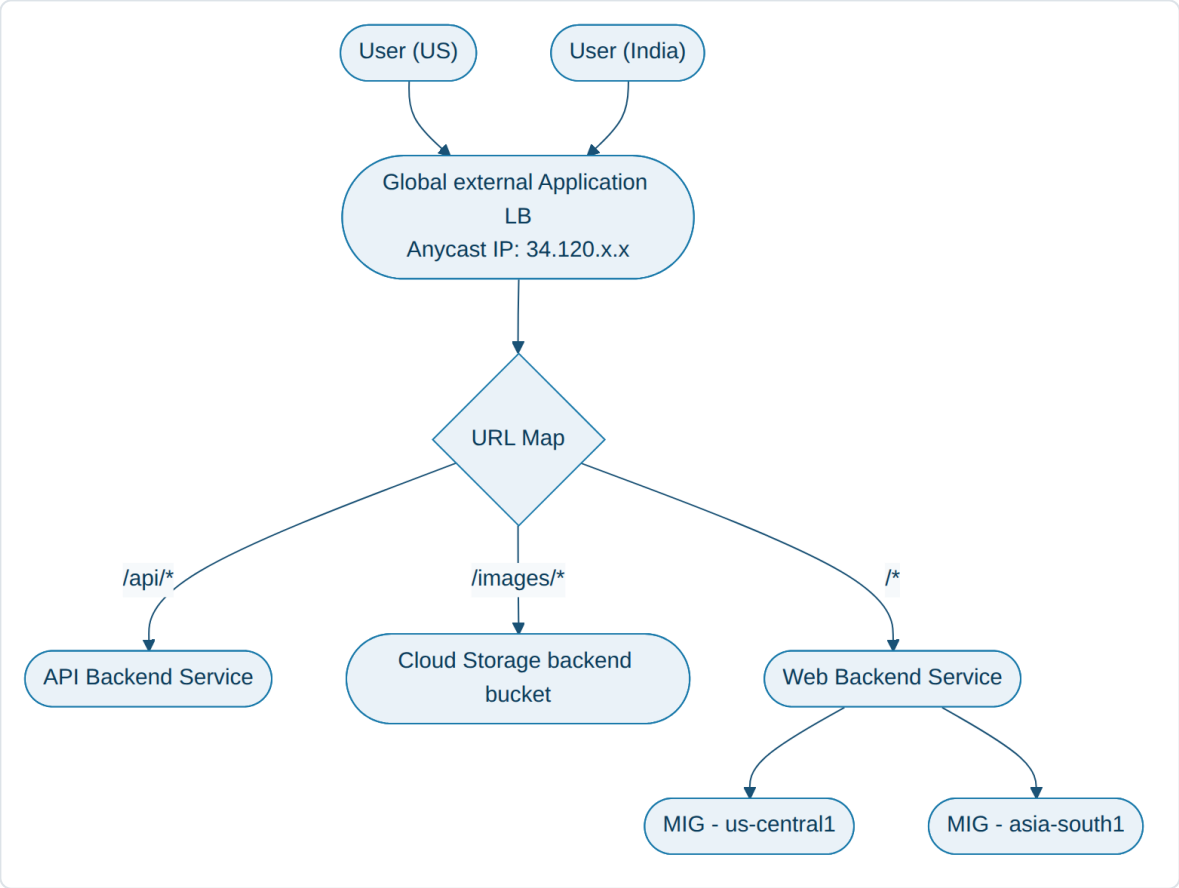
• Common components → Forwarding Rule (IP + port) → Target Proxy → URL Map (routing rules) → Backend Service → Health Check → Backends (Managed Instance Group / GKE NEG / Cloud Run).

Which load balancer should I use?



Example → Global e-commerce site

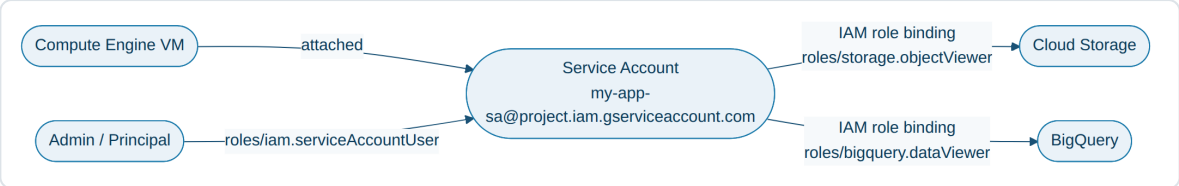
- Backends → MIGs in `us-central1` (US customers) and `asia-south1` (India customers).
- Use → **Global external Application Load Balancer** — one anycast IP (`34.120.x.x`), Google routes each user to the nearest healthy region automatically.
- URL map rules → `/api/*` → API backend service, `/images/*` → Cloud Storage backend bucket, everything else → web backend service.
- Health check → `GET /healthz` every 10s; unhealthy VMs are auto-removed from rotation.



Service Account

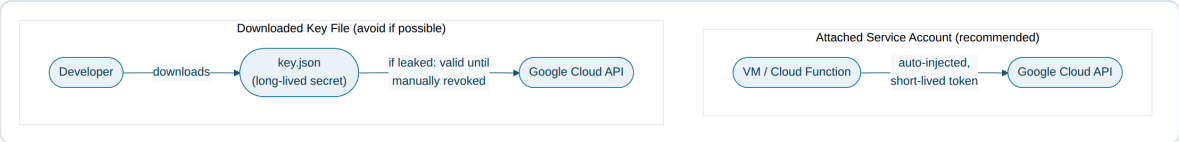
- Service Account (SA) → a special GCP identity used by **applications/workloads** (VMs, GKE pods, Cloud Functions) instead of a human user, to call GCP APIs.
- Every SA has a unique email → `name@project-id.iam.gserviceaccount.com`.
- Two roles a Service Account plays →
- As a **Principal** → you grant it IAM roles so it can access other resources (eg. `roles/storage.objectViewer`).
- As a **Resource** → other principals can be granted roles *on* it (eg. `roles/iam.serviceAccountUser`) so they're allowed to attach/use it.
- Default vs User-managed →
- **Default SA** → auto-created per project (Compute Engine default SA), broad `Editor` access — not recommended for production.
- **User-managed SA** → created explicitly, scoped with least-privilege custom/predefined roles — recommended.
- Best practice → attach a dedicated least-privilege SA to each VM/workload instead of using default SA / broad roles.

Service Account / IAM flow diagram



Attached Service Account vs downloaded Key file

- **Attached SA** (recommended) → GCP automatically injects short-lived credentials into the VM/ workload — no secret file to manage, rotate, or leak.
- **Key file (JSON)** (avoid unless required, e.g. calling GCP from outside GCP) → a long-lived private key downloaded to disk; if leaked, it's valid until manually revoked — bigger security risk.




```
gcloud iam service-accounts create my-app-sa \
  --display-name="My App Service Account"

gcloud projects add-iam-policy-binding my-project \
  --member="serviceAccount:my-app-sa@my-project.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

Example → Cloud Function reading from BigQuery

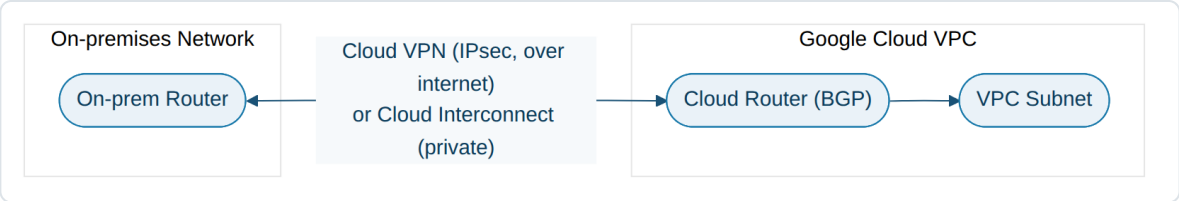
- Cloud Function `generate-report` has attached SA `reports-sa@project.iam.gserviceaccount.com`.
- `reports-sa` is granted `roles/bigquery.dataViewer` + `roles/bigquery.jobUser` (least privilege — read + run query jobs only).
- Function code calls the BigQuery client library with **no credentials in code** — the attached SA's short-lived token is used automatically.
- If someone tries to reuse the function's code outside GCP, it has no valid credentials → fails safely.

Networking → Connectivity Options

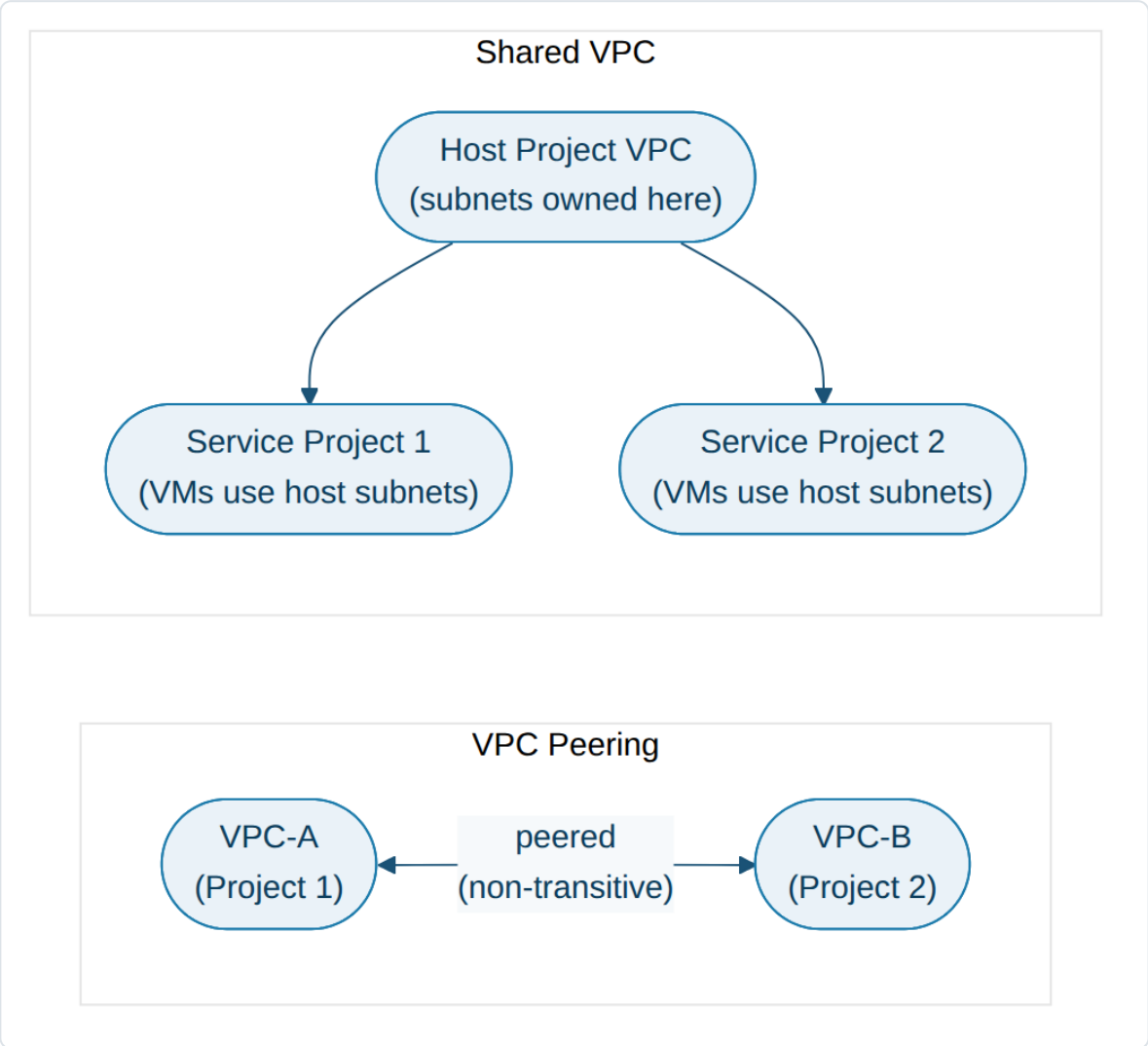
- **VPC Peering** → connects two VPC networks (same or different projects/orgs) privately using internal IPs. Non-transitive (A↔B, B↔C does not mean A↔C).
- **Shared VPC** → a Host project shares its VPC/subnets with one or more Service projects, so all projects' resources sit in one common network (centralized network administration).

- **Cloud VPN** → encrypted IPsec tunnel over the public internet between your VPC and an on-prem/other cloud network.
- **Cloud Interconnect** → a private, high-throughput, low-latency physical (Dedicated) or partner-provided (Partner) connection between on-prem and your VPC (no public internet).
- **Private Google Access** → lets VMs **without an External IP** reach Google APIs/services (Cloud Storage, BigQuery, etc.) using their internal IP, over Google's network.
- **Private Service Connect** → lets you privately consume/publish services across VPC networks using internal IPs (no VPC Peering/public internet needed).

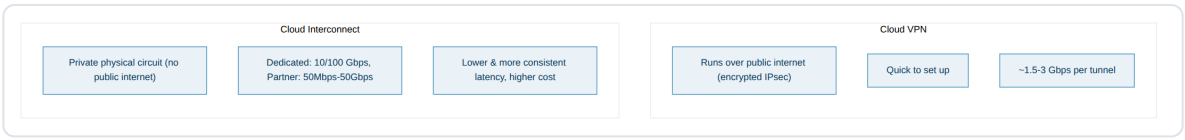
Hybrid connectivity diagram



VPC Peering vs Shared VPC diagram



Cloud VPN vs Cloud Interconnect

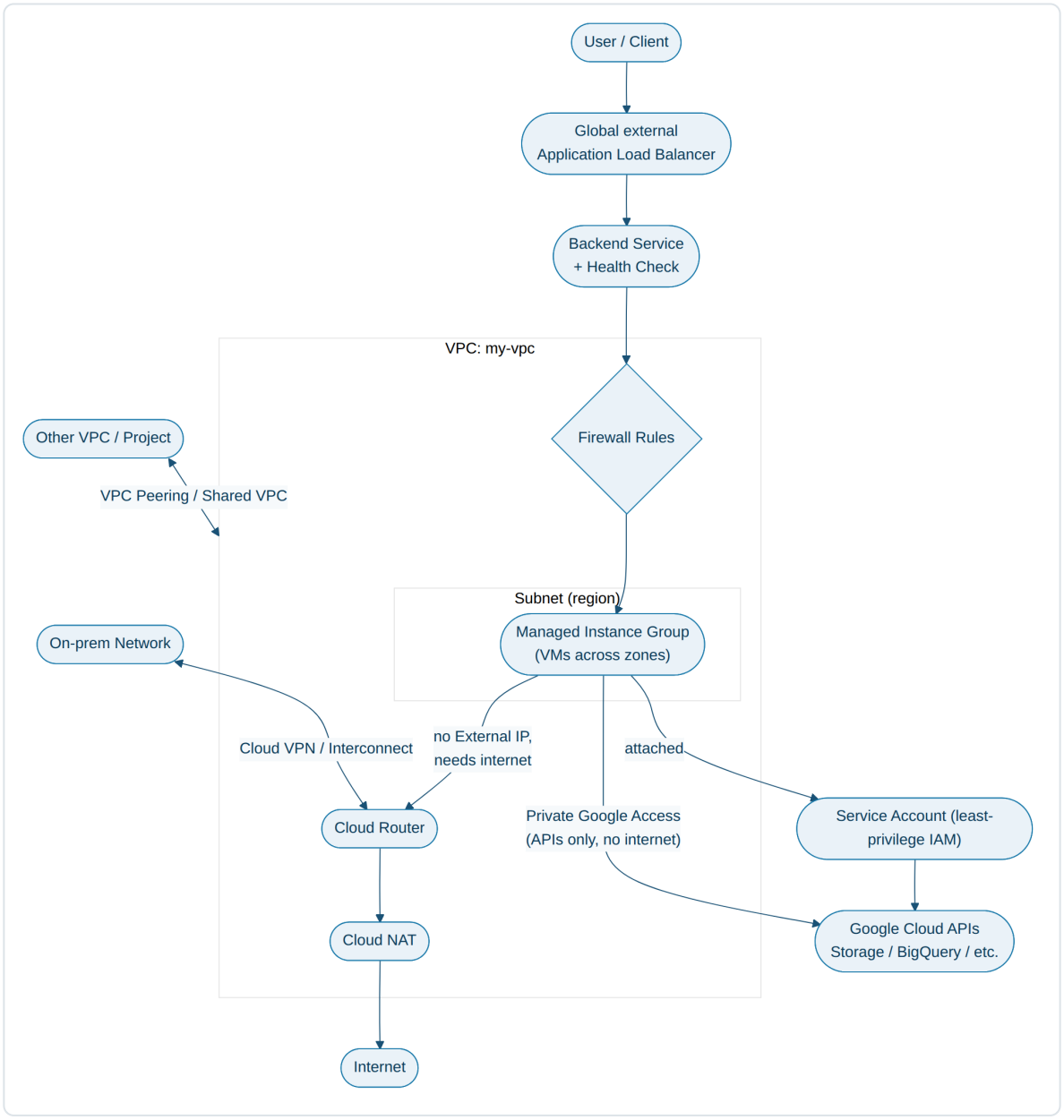


- Cloud VPN → quick to set up, runs over the public internet (encrypted), typically ~1.5–3 Gbps per tunnel, good for smaller/variable workloads or as a backup path.
- Cloud Interconnect → dedicated private circuit, 10 Gbps/100 Gbps per link (Dedicated) or 50 Mbps–50 Gbps (Partner), lower & more consistent latency, higher cost — used for steady, large-scale hybrid traffic.

Example → Hospital chain hybrid connectivity

- On-prem data center hosts the patient records (EHR) system — cannot move to the cloud yet (compliance).
- New analytics workloads run in GCP and need to read on-prem data continuously, in large volume, with low latency.
- Solution → **Dedicated Interconnect** (private, high-throughput) between the hospital's data center and GCP, with **Cloud Router** exchanging routes via BGP.
- A **Cloud VPN** tunnel is also configured as an automatic failover path in case the Interconnect link goes down.

Putting it all together → Full Networking & Compute Architecture



Flow explanation

1. User/Client hits the Global external Application Load Balancer's anycast IP.
2. LB terminates TLS, applies URL map/routing rules, forwards to the nearest healthy Backend Service.
3. Backend Service → Managed Instance Group (VMs across zones) inside a Subnet of the VPC — traffic passes the VPC Firewall Rules first.
4. Each VM has a Service Account attached → used to call other GCP APIs (eg. Cloud Storage, Cloud SQL) with least-privilege IAM roles.
5. If a VM (no External IP) needs outbound internet access (e.g. OS updates, external API call) → traffic goes via Cloud Router + Cloud NAT.

6. If the VM only needs to reach Google APIs (not the public internet) → it uses Private Google Access instead of Cloud NAT.
7. On-prem network reaches the VPC privately via Cloud VPN / Cloud Interconnect, routed through Cloud Router (BGP).
8. Other VPCs (different project/team) connect via VPC Peering, or share this VPC's subnets via Shared VPC.